

Day 05 : OWASP UnCrackable App for Android – Level 3

Reverse Engineering, Anti-Tampering Bypass and Secret Extraction

Name: Vimal Mudalagi

Institution: Digital University Kerala

Programme: M.Sc. Computer Science

Specialization: Cybersecurity

1. Objective

The objective of this task was to analyze the OWASP UnCrackable-Level3 Android application, identify its anti-tampering and anti-analysis protections, bypass those protections using dynamic instrumentation, and extract the hidden secret stored/generated by the native component.

2. Application Details

Item	Details
Application	OWASP UnCrackable Level 3
Package	owasp.mstg.uncrackable3
Version	1.0
Version Code	1
APK	UnCrackable-Level3.apk
SHA-256	6827F776B7D2844342711BE6F13695AE8D1A5F209FC0072DBAF8B85E2FCFF4B
Native Library	libfoo.so
Architectures	ARM64, ARMv7, x86, x86_64
Testing Environment	Android 13 (API 33) emulator
Instrumentation Tool	Frida 17.18.0

3. Tools Used

- Android Debug Bridge (ADB)
- Android Emulator
- Frida 17.18.0
- Frida Server
- PowerShell
- 7-Zip
- APK static inspection

- Native library analysis
- Frida Java and native hooks

4. Application Installation

The APK was installed on the Android emulator using:

```
adb install -r ".\UnCrackable-Level3.apk"
```

Installation completed successfully. The package was confirmed using:

```
adb shell pm path owasp.mstg.uncrackable3
```

The application contained the native library libfoo.so for multiple CPU architectures.

5. Security Protections Identified

The application implements multiple protections intended to make reverse engineering and dynamic analysis more difficult.

5.1 Java-Level Tamper Detection

The application contains Java-side protection logic that detects tampering and can display a warning before terminating execution. The MainActivity contains methods including:

- verifyLibs()
- showDialog()
- init(byte[])
- baz()
- verify(View)

The application therefore performs security-related checks before allowing normal execution.

5.2 Native Frida/Xposed Detection

A major protection mechanism exists inside libfoo.so. The native code searches process/library strings for instrumentation frameworks such as **frida** and **xposed**.

The native implementation uses the C library function `strstr()`. If a suspicious string is detected, the application calls its termination routine (`goodbye()`), causing the process to terminate.

5.3 Native Secret Storage / Generation

The actual secret is not simply present as a plaintext Java string. The application uses native code in libfoo.so to generate/process the secret. For the x86_64 library used by the emulator, the relevant secret-generation function was located at:

```
libfoo.so + 0x12c0
```

5.4 XOR Obfuscation

The generated secret is obfuscated using XOR. The application supplies an XOR key through `MainActivity.init(byte[])`. Therefore, obtaining the encoded native data alone does not immediately reveal the plaintext secret.

6. Frida Setup

The emulator was started with root access and Frida Server was running. Frida connectivity was verified with:

```
frida-ps -U
```

The application was then launched using Frida's spawn mode:

```
frida -U -f owasp.mstg.uncrackable3 -l .\day5_correct.js
```

Spawn mode was important because the required hooks needed to be installed early during application startup.

7. Bypassing Native Frida Detection

The application's native code uses `strstr()` to detect suspicious strings. A Frida hook was therefore placed on the native `strstr()` function:

```
const libc = Process.getModuleByName("libc.so");
const strstr = libc.getExportByName("strstr");

Interceptor.attach(strstr, {
  onEnter(args) {
    this.block = false;
    try {
      const s = args[0].readUtf8String();
      if (s && s.toLowerCase().includes("frida")) {
        this.block = true;
      }
    } catch (e) {}
  },
  onLeave(retval) {
    if (this.block) retval.replace(ptr(0));
  }
});
```

Result

When the native code searched for "frida", the hook forced `strstr()` to return NULL. This prevented the native detection routine from recognizing Frida.

8. Capturing the XOR Key

Instead of attempting to capture an arbitrary `strncpy()` call, the XOR key was captured directly from the application's Java method `MainActivity.init(byte[])`. The method was hooked using:

```
const MainActivity = Java.use("sg.vantagepoint.uncrackable3.MainActivity");
const init = MainActivity.init.overload("[B");

init.implementation = function (key) {
  xorKey = new Uint8Array(key.length);
  for (let i = 0; i < key.length; i++) {
    xorKey[i] = key[i];
  }
  console.log("[+] XOR key captured from init()");
  return init.call(this, key);
};
```

The following output confirmed successful capture:

```
[+] XOR key captured from init()
```

9. Hooking the Secret Generator

The native secret-generation function was located at libfoo.so + 0x12c0. The hook was installed using:

```
const generator = lib.base.add(0x12c0);

Interceptor.attach(generator, {
  onEnter(args) {
    this.location = args[0];
  },
  onLeave(retval) {
    // Read and decode generated secret
  }
});
```

The successful run confirmed:

```
[+] Generator hook installed: 0x7c44bde482c0
```

10. Decoding the Secret

The generated native data consisted of 24 bytes. The captured XOR key was used to decode the data:

```
const encoded = new Uint8Array(this.location.readByteArray(24));
const decoded = new Uint8Array(24);

for (let i = 0; i < 24; i++) {
  decoded[i] = encoded[i] ^ xorKey[i];
}
```

The decoded bytes were then converted into ASCII text.

11. Final Extraction Output

When the application verification function was triggered, Frida produced:

[+] verify() called

[+] SECRET HEX: 6d 61 6b 69 6e 67 20 6f 77 61 73 70 20 67 72 65 61 74 20 61 67 61 69 6e

[+] SECRET TEXT: making owasp great again

Exact Hidden Secret: making owasp great again

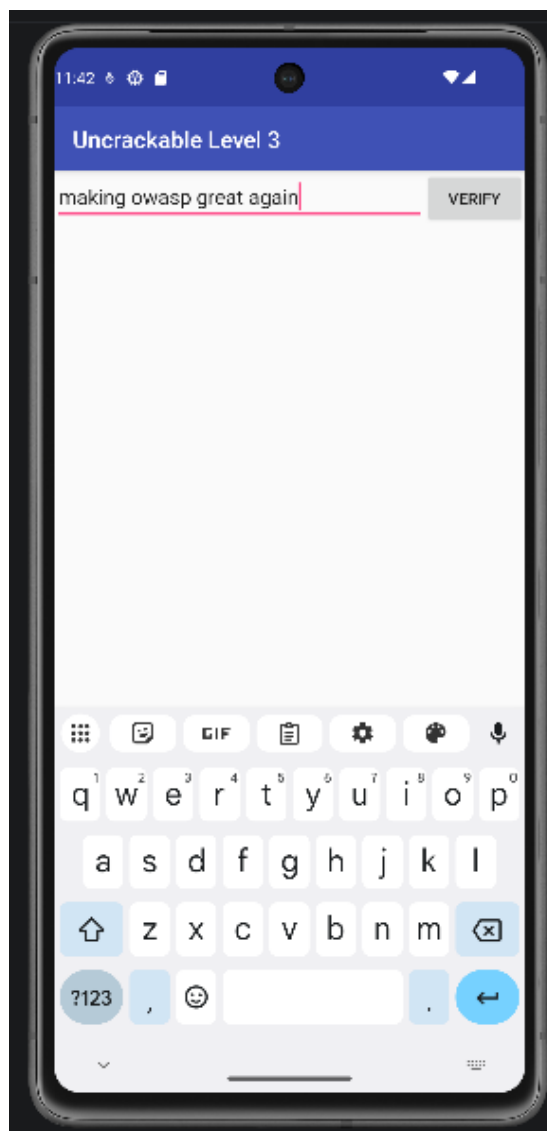


Figure 1: UnCrackable Level 3 app with the decoded secret string 'making owasp great again' entered into the input field.

12. Successful Extraction Workflow

1. Install APK
2. Start rooted Android emulator
3. Start Frida instrumentation
4. Spawn UnCrackable Level 3
5. Bypass native "frida" detection
6. Hook MainActivity.init(byte[])
7. Capture XOR key
8. Locate libfoo.so
9. Hook libfoo.so + 0x12c0
10. Capture generated encoded secret
11. XOR encoded data with captured key
12. Decode 24 bytes
13. Extract plaintext secret

13. Main Frida Techniques Used

Technique	Description
Java method hooking	Used to intercept MainActivity.init(byte[]) and MainActivity.verify(android.view.View)
Native function hooking	Used Interceptor.attach() to intercept native functions
Native API interception	Intercepted libc.so → strstr() to bypass Frida detection
Native function offset hooking	Hooked libfoo.so + 0x12c0 to capture the generated secret
Memory reading	Used readByteArray() to extract the generated bytes from native memory
XOR decoding	The encoded bytes were XORed with the runtime-captured key to recover the plaintext

14. Security Mechanisms Summary

Protection	Location	Bypass Technique
Java tamper checks	MainActivity	Frida Java hooks
Frida detection	libfoo.so	Hook strstr()
Native termination	libfoo.so	Prevent detection condition
Native secret generation	libfoo.so	Hook generator
XOR obfuscation	Native/Java interaction	Capture key through init(byte[])
Runtime verification	MainActivity.verify()	Frida method hook/observation

15. Conclusion

The OWASP UnCrackable Level 3 application uses a combination of Java-level checks, native anti-instrumentation mechanisms, native secret generation, and XOR-based obfuscation to resist dynamic analysis.

Using Frida, the protections were bypassed by intercepting the native strstr() function, capturing the XOR key from MainActivity.init(byte[]), and hooking the native secret-generation function in libfoo.so. The encoded 24-byte secret was successfully extracted from native memory and decoded using the captured XOR key.

Final Hidden Secret: making owasp great again

This exercise demonstrated practical techniques for Android dynamic analysis, Java/native Frida instrumentation, anti-Frida bypass, native memory inspection, and runtime secret extraction.

Appendix A: Full Frida Script (day5_correct.js)

The complete instrumentation script used for this assignment is reproduced below.

```
console.log("[*] Corrected Day 5 script");

const libc = Process.getModuleByName("libc.so");

// Block native Frida detection WITHOUT logging every call
const strstr = libc.getExportByName("strstr");

Interceptor.attach(strstr, {
  onEnter(args) {
    this.block = false;
    try {
      const s = args[0].readUtf8String();
      if (s && s.toLowerCase().includes("frida")) {
        this.block = true;
      }
    } catch (e) {}
  },
  onLeave(retval) {
    if (this.block) retval.replace(ptr(0));
  }
});

let xorKey = null;
const LEN = 24;

// Capture the key from MainActivity.init(byte[])
Java.perform(function () {
  const MainActivity = Java.use("sg.vantagepoint.uncrackable3.MainActivity");

  const init = MainActivity.init.overload("[B");

  init.implementation = function (key) {
    try {
      xorKey = new Uint8Array(key.length);

      for (let i = 0; i < key.length; i++) {
        xorKey[i] = key[i];
      }
    }
  }
});
```

```

    console.log("[+] XOR key captured from init()");
  } catch (e) {
    console.log("[-] Key capture error: " + e);
  }

  return init.call(this, key);
};

console.log("[+] init(byte[]) hook installed");

// Trigger verification once when user presses VERIFY
const verify = MainActivity.verify.overload("android.view.View");

verify.implementation = function (view) {
  console.log("[+] verify() called");
  return verify.call(this, view);
};

console.log("[+] verify() hook installed");
});

// Hook native secret generator
let generatorHooked = false;

const timer = setInterval(function () {
  if (generatorHooked) return;

  try {
    const lib = Process.getModuleByName("libfoo.so");
    generatorHooked = true;
    clearInterval(timer);

    const generator = lib.base.add(0x12c0);

    console.log("[+] Generator hook installed: " + generator);

    Interceptor.attach(generator, {
      onEnter(args) {
        this.location = args[0];
      },

      onLeave(retval) {
        if (xorKey === null) {
          console.log("[-] XOR key not available");
          return;
        }
      }
    });
  } catch (e) {
    console.log("[-] Generator hook failed: " + e);
  }
}, 100);

```

```

    }

    const encoded = new Uint8Array(this.location.readByteArray(LEN));
    const decoded = new Uint8Array(LEN);

    for (let i = 0; i < LEN; i++) {
        decoded[i] = encoded[i] ^ xorKey[i];
    }

    let hex = "";
    for (let i = 0; i < LEN; i++) {
        hex += decoded[i].toString(16).padStart(2, "0") + " ";
    }

    let text = "";
    for (let i = 0; i < LEN; i++) {
        const c = decoded[i];
        text += (c >= 32 && c <= 126) ? String.fromCharCode(c) : ".";
    }

    console.log("[+] SECRET HEX: " + hex);
    console.log("[+] SECRET TEXT: " + text);
    }
});
} catch (e) {}
}, 100);

```

The script was saved and launched using:

```
frida -U -f owasp.mstg.uncrackable3 -l .\day5_correct.js
```